

Binary Search:

Binary Search is an efficient searching algorithm used to find an element in a **sorted array or list**. It works by repeatedly dividing the search interval into two halves until the element is found or the search interval becomes empty.

Algorithm:

BinarySearch(arr, target)

- 1. Start**
- 2. Set low = 0**
- 3. Set high = length(arr) - 1**
- 4. While low <= high:**
 - a. mid = (low + high) // 2**
 - b. If arr[mid] == target:**
Return mid
 - c. Else if target < arr[mid]:**
high = mid - 1
 - d. Else:**
low = mid + 1

5. Return -1 (Element not found)

6. Stop

Out put:

```
Enter the number of elements: 4
Enter the elements:
3
6
8
4

Sorted Array: [3, 4, 6, 8]
Enter the element to search: 4

Element 4 found at index 1.

Execution Time: 0.0000108760 seconds

Time Complexity:
Best Case      : O(1)
Average Case   : O(log n)
Worst Case     : O(log n)
Space Complexity: O(1)

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

- **Best Case: $O(1)$**
The element is found in the first comparison (middle element).
- **Average Case: $O(\log n)$**
The search space is reduced by half in each step.

- **Worst Case: $O(\log n)$**

The element is found after several divisions or is not present in the array.

Space Complexity:

- **Iterative Binary Search: $O(1)$**

Uses only a few variables (low, high, and mid).

- **Recursive Binary Search: $O(\log n)$**

Uses additional memory for the recursive function call stack.